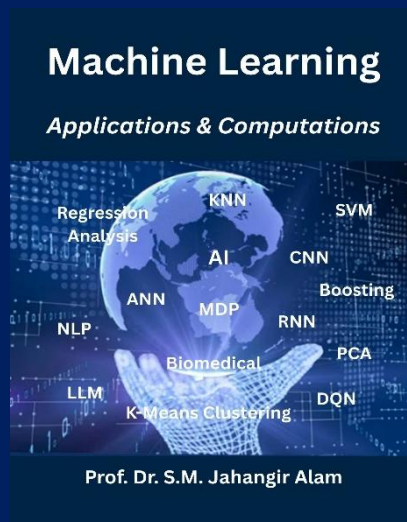


Lecture 5.1

Ensemble Learning Learning



Instructor Name:
Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Ensemble Learning

- **Ensemble Learning** is a powerful ML technique that combines the predictions of **multiple models (learners)** to produce a result that is usually **more accurate and robust** than any single model alone.
- A ML technique where we **combine multiple models (weak learners)** to build a **stronger model**.
- Instead of relying on one model (which might have high variance or bias), ensemble learning merges several models to:
- **Reduce variance** (by averaging out errors),
- **Reduce bias** (by combining diverse models),
- **Improve generalization** on unseen data.

Ensemble Learning

- Instead of relying on one model, use the **wisdom (Knowledge) of the crowd** → combining predictions improves accuracy, stability, and generalization (simplification).
- Bagging → parallel, reduces variance.
- Boosting → sequential, reduces bias.
- Stacking → combines different models with a meta-learner.

Types of Ensemble Learning:

1. Bagging (Bootstrap Aggregating): Good for reducing variance.

- Train models on different random subsets of the data (with replacement).
- Final prediction: average (regression) or majority vote (classification).
- Example: Random Forest (ensemble of decision trees).
- Formula for Bagging (average of models):

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M f_m(x)$$

Types of Ensemble Learning:

2. Boosting (Good for **reducing bias** and build a strong learner from weak ones.)

- Train models sequentially: Each new model focuses on the errors (misclassified points or residuals) of the previous one.
- Final prediction: weighted combination of all models.
- Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost.
- Simplified Gradient Boosting step:

$$F_m(x) = F_{m-1}(x) + \alpha h_m(x)$$

where $h_m(x)$ fits the residuals of the previous model.

Types of Ensemble Learning:

3. Stacking (Stacked Generalization) (Combining different types of models.)

- Train different models (heterogeneous) (e.g., Logistic Regression, Decision Tree, SVM).
- Their predictions are combined by a meta-model (2nd-level learner).
- Example: Predictions from base learners → fed into Logistic Regression as a meta-classifier.

Example:

- Base models → Decision Tree, SVM, kNN
- Meta model → Logistic Regression that combines their predictions.

$$\hat{y} = g(f_1(x), f_2(x), \dots, f_M(x))$$

where, g is the meta-learner.

Ensemble Learning Workflow

1. Train multiple models.
2. Aggregate predictions:
 - **Voting** (classification): majority wins.
 - **Averaging** (regression).
 - **Weighted averaging** (boosting).
3. Final result = improved performance.

Advantages:

- Higher accuracy than individual models.
- Reduces overfitting (especially bagging).
- Works well with unstable learners (like decision trees)
- Works well on both classification and regression
- Robust to noise (especially bagging)

Ensemble Learning Workflow

Disadvantages:

- More computationally expensive.
- Harder to interpret.
- May not help if base models are already strong & highly correlated.

Real-world Applications

- **Random Forests** for image classification or feature importance
- **XGBoost / LightGBM** for Kaggle competitions and finance predictions
- **Stacking** in medical or recommendation systems

Boosting Algorithm

- **Boosting** is an **ensemble learning technique** that builds a **strong classifier** (or regressor) by **combining multiple weak learners** sequentially.
- Each new model **focuses on the mistakes** made by the previous ones to improve overall accuracy.
- **Goal:** Reduce bias and variance.

Core Idea

- Train a weak model $\rightarrow$ see where it fails $\rightarrow$ give more importance (weight) to the misclassified points $\rightarrow$ train next model to fix them.
- Final prediction: weighted vote (classification) or weighted sum (regression).

Boosting works in a series of rounds:

- Train a weak learner (e.g., a small decision tree).
- Increase the weights of the samples that were misclassified — these become more important in the next round.
- Train the next learner to correct the previous learner's errors.
- Combine all learners' outputs through a weighted vote (classification) or weighted sum (regression).
- Over time, the ensemble “boosts” the weak models' performance into a strong model.

Mathematical Idea

Let, $h_m(x)$: prediction of the m -th weak learner, α_m : weight assigned to that learner. The final boosted model:

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

Each new model $h_m(x)$ tries to minimize total error of the combined model $F(x)$.

Ensemble Learning Workflow

Example (Conceptual): Suppose you have a dataset where,

- Model 1 correctly classifies 70% of samples.
- Boosting gives more weight to the 30% misclassified.
- Model 2 learns those hard cases and improves accuracy.
- Model 3 refines the errors further.
- Final combined model achieves **>90% accuracy**.

Popular Boosting Algorithms

Algorithm	Main Idea	Notes
AdaBoost (Adaptive Boosting)	Reweights misclassified samples after each round.	Great for simple datasets, uses exponential loss.
Gradient Boosting	Fits new learners to the residuals (errors) of the previous model.	More flexible and powerful.
XGBoost	Optimized version of Gradient Boosting (regularization + parallelism).	Very fast and accurate.
LightGBM	Uses histogram-based splits for large datasets.	Handles large-scale data efficiently.
CatBoost	Handles categorical features automatically.	Robust and easy to use.

Boosting (AdaBoost)

5.1##A simple Boosting (AdaBoost) illustrating how AdaBoost combines weak classifiers to improve accuracy.

- Scenario: We have a small dataset and two weak classifiers that make simple predictions. We'll combine them using AdaBoost to get a stronger classifier.

Sample	Feature x	True Label y
1	1	+1
2	2	+1
3	3	-1
4	4	-1

Step 1: Initialize sample weights ω_i

- Equal weights initially:

$$\omega_i = \frac{1}{N} = \frac{1}{4} = 0.25$$

Boosting (AdaBoost)

Step 2: First weak classifier

$h_1(x)$:

- Rule: Predict +1 if $x < 3$,
else -1

Sample	x	y	$h_1(x)$	Correct?
1	1	+1	+1	Yes
2	2	+1	+1	Yes
3	3	-1	-1	Yes
4	4	-1	-1	Yes

Error $\epsilon_1 = 0$ (perfect classifier).

Since $\epsilon_1 = 0$, AdaBoost stops here (perfect classifier). But let's pretend h_1 made some errors for illustration.

Modified $h_1(x)$: Predict +1 if $x \leq 2$

Sample	X	Y	$h_1(x)$	Correct?
1	1	+1	+1	Yes
2	2	+1	+1	Yes
3	3	-1	+1	No
4	4	-1	-1	Yes

Boosting (AdaBoost)

Step 3: Calculate error of $h_1(x)$

$$\epsilon_1 = \sum \omega_i \text{ for wrong predictions} = 0.25$$

Step 4: Calculate classifier weight α

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right) = \frac{1}{2} \ln \left(\frac{0.75}{0.25} \right) = 0.5493$$

Step 5: Update sample weights ω_i : For each sample:

$$\omega_i^{new} \leftarrow \omega_i \cdot e^{-\alpha y_i h_1(x_i)}$$

• If correctly classified $y_i = h_1(x_i)$:

$$\omega_1^{new} = \omega_1 \cdot e^{-\alpha y_1 h_1(x_1)} = 0.25 \times e^{-(0.5493)(+1)(+1)} = 0.144$$

• If misclassified: multiply by e^α

$$\omega_3^{new} = \omega_1 \cdot e^{-\alpha y_1 h_1(x_1)} = 0.25 \times e^{-(0.5493)(-1)(+1)} = 0.433$$

Boosting (AdaBoost)

Sample	y_i	$h_1(x_i)$	Correct?	Old ω_i	Update factor	New ω_i
1	+1	+1	Yes	0.25	$e^{-0.5493}=0.577$	$0.25 \times 0.577 = 0.144$
2	+1	+1	Yes	0.25	$e^{-0.5493}=0.577$	$0.25 \times 0.577 = 0.144$
3	-1	+1	No	0.25	$e^{0.5493}=1.732$	$0.25 \times 1.732 = 0.433$
4	-1	-1	Yes	0.25	$e^{0.5493}=0.577$	$0.25 \times 0.577 = 0.144$

Boosting (AdaBoost)

- **Step 6:** Normalize weights

Sum: $0.144 + 0.144 + 0.433 + 0.144 = 0.865$

Sample	New ω_i	New Weight (Normalized ω_i)
1	0.144	$0.144/0.865=0.1665$
2	0.144	$0.144/0.865=0.1665$
3	0.433	$0.433/0.865=0.500$
4	0.144	$0.144/0.865=0.1665$

Boosting (AdaBoost)

Step 7: Second weak classifier $h_2(x)$

- Rule: Predict +1 if $x < 4$, else -1

Sample	x	y	$h_2(x)$	Correct?
1	1	+1	+1	Yes
2	2	+1	+1	Yes
3	3	-1	+1	No
4	4	-1	-1	Yes

Calculate error weighted by new weights:

$$\epsilon_2 = \sum \text{misclassified } \omega_i = \omega_3 = 0.5$$

It means that the classifier is no better than random guessing, so AdaBoost would stop or pick a different classifier.

Boosting (AdaBoost)

Step 7: Compute α_2 :

$$\begin{aligned}\alpha_2 &= \frac{1}{2} \ln \frac{1 - \epsilon_1}{\epsilon_1} \\ &= \frac{1}{2} \ln \frac{1 - 0.1665}{0.1665} = 0.8053\end{aligned}$$

Sample	x	y	$h_2(x)$	Correct?
1	1	+1	+1	Yes
2	2	+1	+1	Yes
3	3	-1	+1	No
4	4	-1	-1	Yes

Step 8: Final strong classifier:

$$H(x) = \text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x))$$

Step 9: Predict for $X = 3$: $h_1(3) = +1$, $h_2(3) = -1$

Calculate: $\alpha_1 \times h_1(3) + \alpha_2 \times h_2(3) = 0.5493 \times 1 + 0.8053 \times (-1) = -0.256$

$$\text{sign}(x) = \begin{cases} +1 & \text{for } x > 0 \\ 0 & \text{for } x = 0 \\ -1 & \text{for } x < 0 \end{cases}$$

Prediction:
 $\text{sign}(-0.256) = -1$

Boosting (AdaBoost)

Summary:

- AdaBoost adjusts weights to focus on misclassified samples.
- Classifier weights α indicate confidence.
- Final prediction is a weighted vote of weak classifiers.

Step	Description
Initialize weights	Equal weights for all samples
Train weak learner	Minimize weighted error
Compute α	Weight of learner based on error
Update weights	Increase weight on misclassified samples
Repeat	Train next learner on updated weights
Final prediction	Weighted majority vote of learners

Gradient Boosted Trees

- Gradient Boosting builds an ensemble of decision trees sequentially, where each new tree tries to correct the errors made by the previous one.
- Gradient Boosted Trees (GBT or GBM) are an **ensemble learning method** that combines many **weak learners** (usually decision trees) into a **strong model**.
- **Boosting:** Build trees sequentially, where each new tree corrects the errors of the previous trees.
- Instead of just counting mistakes (like AdaBoost), Gradient Boosting uses gradients (from calculus) to minimize a chosen loss function (e.g., **squared error, log loss**).
- Gradient Boosted Trees: build trees one by one, each learning from the errors of the previous, using gradient descent on the loss function.

Gradient Boosted Trees

Key Idea:

1. Start with a **weak model** (often just a constant prediction, like the mean of labels).
2. Compute the **errors/residuals**.
3. Fit a new **decision tree** to predict those errors.
4. Update the model by adding this new tree (scaled by learning rate).
5. Repeat until convergence or max trees are reached.

Key Concepts:

- **Start** with a constant model
- **Compute residuals** (errors)
- **Fit a new tree** to residuals
- **Update model** using learning rate
- **Repeat** until convergence or max trees

Gradient Boosted Trees (Algorithm (Regression))

1. Initialize model with a constant:

$$F_0(x) = \arg \min_{\gamma} \sum L(y_i, \gamma)$$

- For squared error loss, it's just the mean of y .

2. For each boosting step $m = 1 \dots M$:

- Compute gradient (residuals):

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right] \quad F(x) = F_{m-1}(x)$$

- Fit a regression tree $h_m(x)$ to predict residuals.
- Find optimal multiplier (step size) γ_m .
- Update model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot \gamma_m(x) h_m(x)$$

Where, η = learning rate.

Gradient Boosted Trees

5.6##A Gradient Boosted Trees using a tiny dataset keeping it small and intuitive so the core logic of boosting is clear.

Goal: Predict simple output Y from input X

Dataset:

x	1	2	3	4	5
y	2	3	4	5	6

Clearly, a perfect fit is: $y=x+1$

We'll use this to manually build a Gradient Boosted Regressor.

Step 1: Initialize with a constant prediction

We start with a model that always predicts the **mean of y** :

$$y_{mean} = \frac{2 + 3 + 4 + 5 + 6}{5} = 4$$

Let this be **model $F(x) = 4$** for all x

Gradient Boosted Trees

Step 2: Compute residuals (errors)

$$r_i = y_i - F_0(x_i)$$

Step 3: Fit a small regression tree to residuals

Let's build a **decision stump** (tree with 1 split): We'll split on $x = 3$

- If $x < 3$: predict **-1.5**
- If $x \geq 3$: predict **+1.0**

Let this tree be $h_1(x)$

x	y	$F_0(x_i)$	Residual r
1	2	4	-2
2	3	4	-1
3	4	4	0
4	5	4	+1
5	6	4	+2

Step 4: Update the model

Learning rate $\eta = 0.1$

$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$

Therefore:

- $x = 1,2$: $F_1(x) = 4 + 0.1 \cdot (-1.5) = 4 - 0.15 = 3.85$
- $x = 3,4,5$: $F_1(x) = 4 + 0.1 \cdot (1.0) = 4 + 0.1 = 4.1$

Gradient Boosted Trees

Step 5: New Residuals (after 1st iteration)

x	y	$F_1(x)$	Residual
1	2	3.85	-1.85
2	3	3.85	-0.85
3	4	4.1	-0.1
4	5	4.1	+0.9
5	6	4.1	+1.9

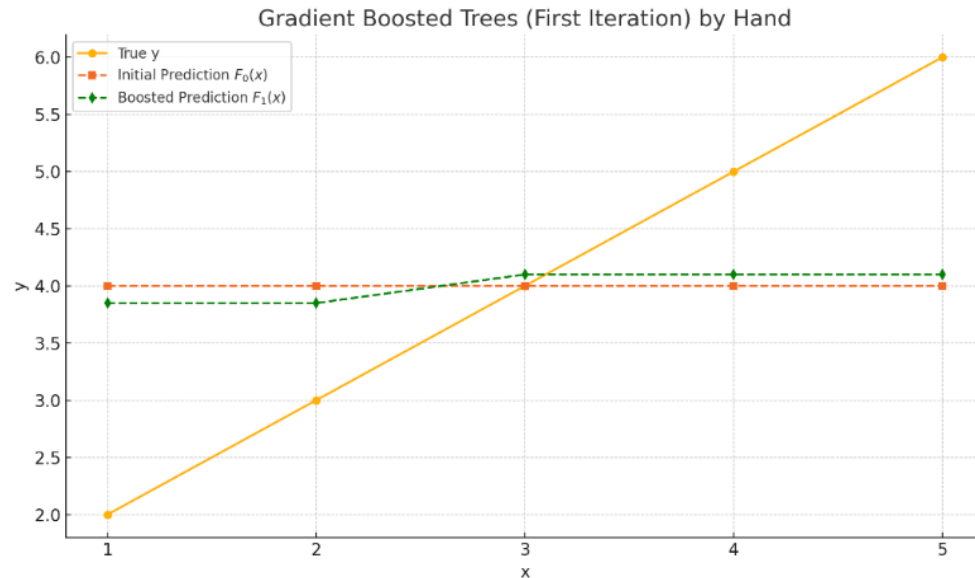
→ Now repeat steps 3–5 with these residuals to build the next tree, and so on.

Summary (After 1 Tree):

x	True y	Prediction $F_1(x)$	Error
1	2	3.85	-1.85
2	3	3.85	-0.85
3	4	4.1	+0.1
4	5	4.1	+0.9
5	6	4.1	+1.9

The model is improving the prediction **gradually** by adding **small trees** that **correct errors**.

Gradient Boosted Trees



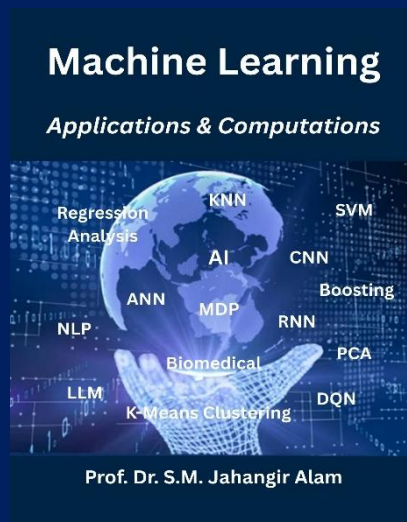
A plot showing the first step of Gradient Boosted Trees:

- Circles = True values
- Squares = Initial model prediction $F_0(x) = 4$
- Diamonds = First boosted prediction $F_1(x)$ after fitting a tree to residuals

You can see how the model begins to shift closer to the actual data using a small tree correction.

Lecture 5.2

Ensemble Learning Learning



Instructor Name:
Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

XGBoost Algorithm

- XGBoost (Extreme Gradient Boosting) is a powerful and popular ensemble learning algorithm based on Gradient Boosting, designed for speed and performance.
- It's widely used in machine learning competitions and real-world applications such as classification, regression, and ranking problems.
- XGBoost builds an ensemble (a collection) of weak learners—usually decision trees—in a sequential manner.
- Each new tree is trained to correct the errors made by the previous ones.

XGBoost Algorithm

- **Initialize the model** with a base prediction (e.g., mean value for regression).

- **Compute residuals (errors):**

$$r_i = y_i - \hat{y}$$

- **Fit a new tree** to predict these residuals.

- **Update predictions:**

$$\hat{y}_i^{(new)} = \hat{y}_i^{(old)} + \eta \cdot f_k(x_i)$$

- where $\eta = \text{learning rate}$
 - $f_k(x_i)$ = prediction from the new tree
- **Repeat** the process until a stopping criterion is met (e.g., number of trees or minimum loss improvement).

XGBoost Algorithm: Objective Function

- The model minimizes an objective function combining loss and regularization:

$$Obj = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Where,

- $l(y_i, \hat{y}_i)$ = loss function (e.g., squared error, log loss)
- $\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$ (regularization to prevent overfitting)

Key Features

- Regularization** – Both L1 (Lasso) and L2 (Ridge) penalties to reduce overfitting
- Parallel Processing** – Uses CPU cores efficiently
- Handling Missing Values** – Automatically learns the best way to handle missing data
- Tree Pruning** – Uses a more efficient “*max depth*” approach
- Weighted Quantile Sketch** – Handles unbalanced data
- Sparsity Awareness** – Optimized for sparse input (many zeros)

XGBoost Algorithm: Applications

Applications

- Fraud detection
- Medical diagnosis
- Credit scoring
- Sales prediction
- Ranking/recommendation systems
- Kaggle competitions

XGBoost Algorithm: Example

5.7##A ranking/recommendation using XGBoost.

Step 1: Small dataset

Suppose we want to recommend **products to users** based on features. Our dataset has **UserID**, **ProductID**, **Features**, and a **relevance score** (higher = more relevant).

User	Product	Feature (Price)	Feature (Rating)	Relevance
1	A	10	5	3
1	B	20	4	2
1	C	15	3	1
2	D	5	5	2
2	E	15	4	3

Goal: Predict a score for each product for each user, then **rank products**.

XGBoost Algorithm: Example

User	Product	Feature (Price)	Feature (Rating)	Relevance
1	A	10	5	3
1	B	20	4	2
1	C	15	3	1
2	D	5	5	2
2	E	15	4	3

Step 2: Initialize predictions

XGBoost starts with a **base score**. In ranking tasks, we often start with **0**:

$$\hat{y}_0 = 0$$

Initial predictions for all items:

User	Product	Prediction
1	A	0
1	B	0
1	C	0
2	D	0
2	E	0

XGBoost Algorithm: Example

User	Product	Feature (Price)	Feature (Rating)	Relevance
1	A	10	5	3
1	B	20	4	2
1	C	15	3	1
2	D	5	5	2
2	E	15	4	3

Step 3: Create pairwise comparisons

For **pairwise ranking**, we compare **pairs of products for the same user**:

- User 1: (A vs B), (A vs C), (B vs C)
- User 2: (D vs E)

User	Product	Prediction
1	A	0
1	B	0
1	C	0
2	D	0
2	E	0

The **target for pairwise ranking** is:

if item1 relevance > item2 relevance $\rightarrow$ target = 1, else 0

Pairs for User 1:

XGBoost Algorithm: Example

User	Product	Feature (Price)	Feature (Rating)	Relevance
1	A	10	5	3
1	B	20	4	2
1	C	15	3	1
2	D	5	5	2
2	E	15	4	3

The target for pairwise ranking is:

if item1 relevance > item2 relevance $\rightarrow$ target = 1, else 0

Pairs for User 1:

Pair	Target (item1 > item2)
A vs B	1 (3 > 2)
A vs C	1 (3 > 1)
B vs C	1 (2 > 1)

Pairs for User 2:

Pair	Target
D vs E	0 (2 < 3)

User	Product	Prediction
1	A	0
1	B	0
1	C	0
2	D	0
2	E	0

XGBoost Algorithm: Example

Step 4: Compute pseudo-residuals

XGBoost uses **gradient of logistic loss** for ranking. For simplicity, let's approximate:

$$r = Target - \sigma(\hat{y}_i - \hat{y}_j)$$

Where σ is sigmoid:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Since initial predictions are all 0:

$$\sigma(\hat{y}_i - \hat{y}_j) = \sigma(0 - 0) = \sigma(0) = 0.5$$

Residuals for each pair:

- A vs B $\rightarrow r = Target - \sigma(\hat{y}_i - \hat{y}_j) = 1 - 0.5 = 0.5$
- A vs C $\rightarrow r = 1 - 0.5 = 0.5$
- B vs C $\rightarrow r = 1 - 0.5 = 0.5$
- D vs E $\rightarrow r = 0 - 0.5 = -0.5$

XGBoost Algorithm: Example

Step 5: Fit a tree to residuals

We now build a simple **decision stump**.

Let's split on **Rating ≥ 4** :

- **Left node (Rating ≥ 4):**

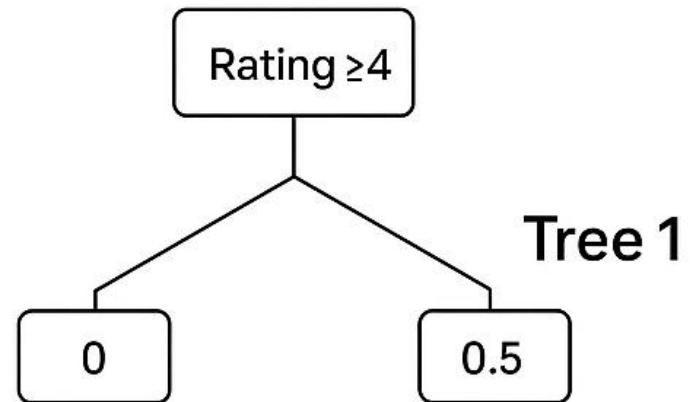
A, B, D, E $\rightarrow$ residuals = 0.5, 0.5, -0.5, -0.5 $\rightarrow$ average = 0

- **Right node (Rating < 4):**

C $\rightarrow$ residual = 0.5 $\rightarrow$ average = 0.5

Tree predicts:

- Rating $\geq 4 \rightarrow 0$
- Rating $< 4 \rightarrow 0.5$



XGBoost Algorithm: Example

Step 6: Update predictions

Use learning rate $\eta = 0.1$:

$$\hat{y}_1 = \hat{y}_0 + \eta \cdot \text{tree prediction}$$

Predictions:

Product	Feature Rating	Prediction
A	5	0
B	4	0
C	3	$0 + 0.1 * 0.5 = 0.05$
D	5	0
E	4	0

Step 7: Rank products for each user

- **User 1:** Predictions:

$A=0, B=0, C=0.05 \rightarrow \text{Rank: } C > A = B \rightarrow \text{not perfect yet}$

- **User 2:** Predictions: $D=0, E=0 \rightarrow \text{Tie}$

XGBoost Algorithm: Example

Summary of Steps for XGBoost Ranking:

1. Start with **initial prediction** (usually 0).
2. Generate **pairwise comparisons** for each user.
3. Compute **pseudo-residuals** for each pair.
4. Fit a **small tree** to residuals.
5. Update predictions using **learning rate**.
6. Repeat for multiple trees until ranking improves.

XGBoost Algorithm: Example

For full 2-tree giving the correct ranking for both users:

Step 1: Current predictions after 1st tree

User	Product	Feature Rating	Prediction
1	A	5	0
1	B	4	0
1	C	3	0.05
2	D	5	0
2	E	4	0

Current ranks (by prediction):

- User 1: $C > A = B \rightarrow$ **wrong**
- User 2: $D = E \rightarrow$ **tie**

We need the second tree to **correct these mistakes**.

XGBoost Algorithm: Example

Step 2: Compute residuals for second tree: compute residuals,

$$r = Target - \sigma(\hat{y}_i - \hat{y}_j)$$

Where σ is sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

User 1 pairs

- A vs B $\rightarrow$ Target=1, Pred diff = 0 - 0 = 0 $\rightarrow \sigma(0)=0.5 \rightarrow r = 1 - 0.5 = 0.5$
- A vs C $\rightarrow$ Target=1, Pred diff = 0 - 0.05 = -0.05 $\rightarrow \sigma(-0.05) \approx 0.4875 \rightarrow r = 1 - 0.4875 \approx 0.5125$
- B vs C $\rightarrow$ Target=1, Pred diff = 0 - 0.05 = -0.05 $\rightarrow \sigma(-0.05) \approx 0.4875 \rightarrow r \approx 0.5125$

User 2 pair

D vs E $\rightarrow$ Target=0, Pred diff = 0 - 0 = 0 $\rightarrow \sigma(0)=0.5 \rightarrow r = 0 - 0.5 = -0.5$

XGBoost Algorithm: Example

Step 3: Fit second tree

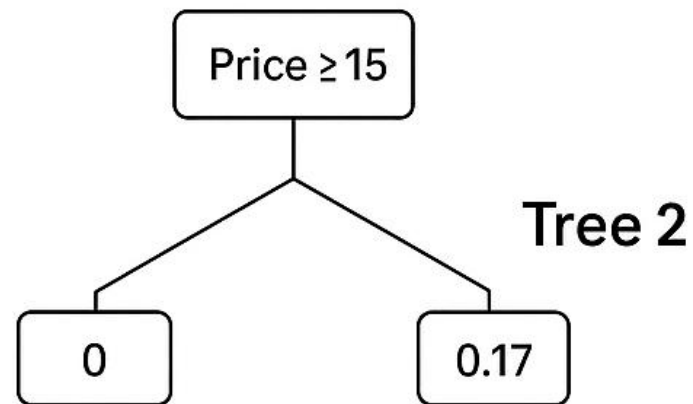
We can split on **Price** ≥ 15 this time:

Product	Price	Residual approximation
A	10	0.5
B	20	0.5
C	15	0.5125
D	5	-0.5
E	15	-0.5

- Left node (Price < 15) $\rightarrow$ A, D $\rightarrow$ residuals = 0.5, -0.5 $\rightarrow$ avg = 0
- Right node (Price ≥ 15) $\rightarrow$ B, C, E $\rightarrow$ residuals = 0.5, 0.5125, -0.5 $\rightarrow$ avg ≈ 0.1708

Tree predicts:

- Price $< 15 \rightarrow 0$
- Price $\geq 15 \rightarrow 0.17$



XGBoost Algorithm: Example

Step 4: Update predictions (learning rate $\eta=0.1$)

$$\hat{y}_2 = \hat{y}_1 + 0.1 \times \text{tree prediction}$$

Product	Previous Prediction	Tree Prediction	Updated Prediction
A	0	0	0
B	0	0.17	$0 + 0.017 = 0.017$
C	0.05	0.17	$0.05 + 0.017 = 0.067$
D	0	0	0
E	0	0.17	$0 + 0.017 = 0.017$

Step 5: Check rankings

User 1

- Predictions: A=0, B=0.017, C=0.067
- Rank: **C > B > A** → Closer to correct (actual relevance: A=3, B=2, C=1)

User 2

- Predictions: D=0, E=0.017
- Rank: **E > D** → Correct (actual relevance: E=3, D=2)

XGBoost Algorithm: Example

Step 6: Key Takeaways

1. Pairwise ranking compares items for each user.
2. Residuals are based on the difference between predicted scores and actual preference.
3. Trees are fit to residuals, and predictions are updated with a learning rate.
4. Multiple trees are required to converge to the correct ranking.

XGBoost Algorithm: Example

For full 3-tree giving the correct ranking for both users:

Step 0:

Dataset

We want
**predicted
scores** to rank
products
correctly for
each user.

User	Product	Price	Rating	Relevance
1	A	10	5	3
1	B	20	4	2
1	C	15	3	1
2	D	5	5	2
2	E	15	4	3

Step 1: Initial prediction (tree 0)

- Base prediction: 0 for all items
- Predictions:

Product	Prediction
A	0
B	0
C	0
D	0
E	0

XGBoost Algorithm: Example

Step 2: First tree (tree 1)

Split: $\text{Rating} \geq 4$

- Left node ($\text{Rating} \geq 4$): A, B, D, E $\rightarrow$ avg residual ≈ 0
- Right node ($\text{Rating} < 4$): C $\rightarrow$ avg residual ≈ 0.5

Learning rate $\eta = 0.1$ $\rightarrow$ Updated predictions:

Rankings after tree1:

- User1: $C > A = B \rightarrow$ still wrong
- User2: $D = E \rightarrow$ tie

Product	Pred (tree1)
A	0
B	0
C	0.05
D	0
E	0

XGBoost Algorithm: Example

Step 3: Second tree (tree 2)

Split: Price ≥ 15

- Left node (Price < 15): A, D $\rightarrow$ avg residual = 0
- Right node (Price ≥ 15): B, C, E $\rightarrow$ avg residual ≈ 0.17

Update predictions:

Product	Previous	Tree prediction	Updated
A	0	0	0
B	0	0.17	0.017
C	0.05	0.17	0.067
D	0	0	0
E	0	0.17	0.017

Rankings after tree2:

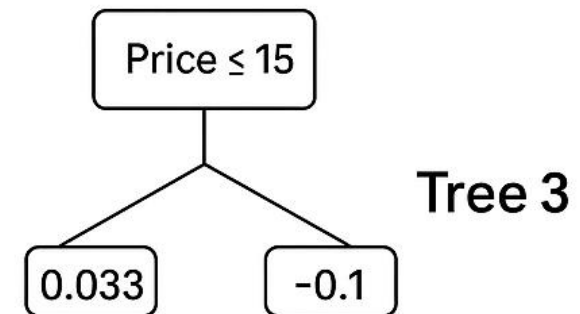
- User1: C $>$ B $>$ A $\rightarrow$ improving but not perfect
- User2: E $>$ D $\rightarrow$ correct

XGBoost Algorithm: Example

Step 4: Third tree (tree 3)

Now we want to fix **User1 ranking**, so we split on **Price ≤ 15** :

- Left node (Price ≤ 15): A, C, D $\rightarrow$ residuals (approx) = 0.33
- Right node (Price > 15): B, E $\rightarrow$ residuals (approx) = -0.1



Update predictions with $\eta = 0.1$:

Product	Previous	Tree prediction	Updated
A	0	0.033	0.033
B	0.017	-0.01	0.007
C	0.067	0.033	0.1
D	0	0.033	0.033
E	0.017	-0.01	0.007

XGBoost Algorithm: Example

Step 5: Final rankings after 3 trees

User 1 predictions:

- . $A = 0.033$
- . $B = 0.007$
- . $C = 0.1$

Rank: $C > A > B \rightarrow$ corresponds to actual relevance $3 > 2 > 1$

User 2 predictions:

- . $D = 0.033$
- . $E = 0.007$

Rank: $D > E \rightarrow$ corresponds to actual relevance $3 > 2$

XGBoost Algorithm: Example

Step 6: Key observations

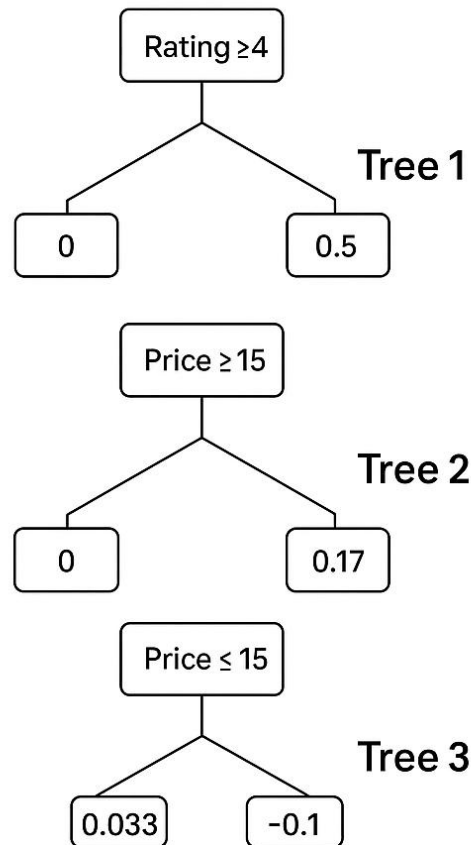
1. XGBoost **gradually adjusts predictions** with small learning steps.
2. Each tree **fits residuals** (pseudo-gradients) to improve ranking.
3. **Pairwise ranking** allows the model to focus on ordering items correctly, not exact values.
4. More trees $\rightarrow$ better ranking, especially with small η .

XGBoost Algorithm: Example

Now both users have **corrected ranking** after 3 trees.

3-Tree XGBoost Ranking Example

User	Product	Price	Relevance
1	A	10	5
2	B	20	4
3	C	15	3
4	D	5	1



Product	Prediction
A	0
B	0.05
C	0.17
D	0.07
E	0.17

Product	Prediction
A	0
B	0.017
C	0.067
D	0
E	0.017

Bagging Algorithm (Bootstrap Aggregating)

Bagging is an **ensemble learning** method used to improve the stability and accuracy of machine learning models (mainly decision trees).

The key idea:

- Train many models on **different random samples** of the training data, then **combine their predictions** (majority vote for classification, average for regression).
- **Bagging**: train multiple models on different bootstrapped samples and aggregate their predictions → reduces variance and improves accuracy.

Special Case:

- Random Forest: Bagging + Random Feature Selection.

Steps of Bagging Algorithm

1. Bootstrap sampling

- From the original dataset (size N), draw N samples with replacement.
- This creates a new dataset called a bootstrap sample.
- Do this B times $\rightarrow$ you get B datasets.

2. Train models

- On each bootstrap sample, train a base learner (e.g., decision tree).
- Each model is slightly different because it sees different data.

3. Aggregate (collective/cumulative) predictions

- Regression $\rightarrow$ take the average of all model predictions.
- Classification $\rightarrow$ take the majority vote of all model predictions.

Steps of Bagging Algorithm

Advantages:

- Reduces variance (especially good for unstable models like decision trees).
- Simple and parallelizable.

Disadvantages:

- Less effective on stable models (e.g., linear regression).
- Larger computation (since many models are trained).

Steps of Bagging Algorithm

5.7##A detailed Bagging algorithm with simple data.

Dataset (6 samples):

ID	1	2	3	4	5	6
Feature (X)	2	4	5	7	8	9
Label (Y)	0	0	1	1	1	0

Goal:

Build an ensemble of 3 classifiers using **Bagging** with bootstrap samples, and predict the label of a new sample $X=6$.

Step 1: Create Bootstrap Samples (Sample with replacement)

Learner	Bootstrap Sample (IDs)
M1	1, 2, 2, 4, 5, 6
M2	2, 3, 3, 5, 5, 6
M3	1, 3, 4, 4, 5, 6

Steps of Bagging Algorithm

Step 2: Train Base Learners

Let's assume each learner is a simple threshold classifier that splits at some X value to predict 0 or 1. We'll pick a simple decision stump for each learner based on its bootstrap sample:

ID	1	2	3	4	5	6
Feature (X)	2	4	5	7	8	9
Label (Y)	0	0	1	1	1	0

Learner M1:

Bootstrap data (IDs): 1(2,0), 2(4,0), 2(4,0), 4(7,1), 5(8,1), 6(9,0)

- Majority labels:
 - ✓ $X \leq 4$: IDs 1 and 2 $\rightarrow$ Labels mostly 0
 - ✓ $X > 4$: IDs 4,5,6 $\rightarrow$ Labels: 1,1,0 $\rightarrow$ Majority 1

Decision stump:

- Predict 0 if $X \leq 4$
- Predict 1 if $X > 4$

Learner	Bootstrap Sample (IDs)
M1	1, 2, 2, 4, 5, 6
M2	2, 3, 3, 5, 5, 6
M3	1, 3, 4, 4, 5, 6

Steps of Bagging Algorithm

ID	1	2	3	4	5	6
Feature (X)	2	4	5	7	8	9
Label (Y)	0	0	1	1	1	0

Learner	Bootstrap Sample (IDs)
M1	1, 2, 2, 4, 5, 6
M2	2, 3, 3, 5, 5, 6
M3	1, 3, 4, 4, 5, 6

Learner M2:

Bootstrap data (IDs): 2(4,0), 3(5,1), 3(5,1), 5(8,1), 5(8,1), 6(9,0)

- ✓ $X \leq 5$: IDs 2,3,3 $\rightarrow$ Labels 0,1,1 $\rightarrow$ Majority 1
- ✓ $X > 5$: IDs 5,5,6 $\rightarrow$ Labels 1,1,0 $\rightarrow$ Majority 1

Decision stump:

Predict 1 for all X (since majority is 1 everywhere)

Steps of Bagging Algorithm

ID	1	2	3	4	5	6
Feature (X)	2	4	5	7	8	9
Label (Y)	0	0	1	1	1	0

Learner	Bootstrap Sample (IDs)
M1	1, 2, 2, 4, 5, 6
M2	2, 3, 3, 5, 5, 6
M3	1, 3, 4, 4, 5, 6

Learner M3:

Bootstrap data (IDs): 1(2,0), 3(5,1), 4(7,1), 4(7,1), 5(8,1), 6(9,0)

- ✓ $X \leq 5$: IDs 1,3 $\rightarrow$ Labels 0,1 $\rightarrow$ Tie, pick 1
- ✓ $X > 5$: IDs 4,4,5,6 $\rightarrow$ Labels 1,1,1,0 $\rightarrow$ Majority 1

Decision stump:

Predict 1 for all X (including $X \leq 5$)

Steps of Bagging Algorithm

ID	1	2	3	4	5	6
Feature (X)	2	4	5	7	8	9
Label (Y)	0	0	1	1	1	0

Learner	Bootstrap Sample (IDs)
M1	1, 2, 2, 4, 5, 6
M2	2, 3, 3, 5, 5, 6
M3	1, 3, 4, 4, 5, 6

Step 3: Predict new sample X=6

Learner	Prediction
M1	$X = 6 > 4 \Rightarrow 1$
M2	Predict 1
M3	Predict 1

Step 4: Aggregate predictions (Majority voting)

Predictions=[1,1,1]

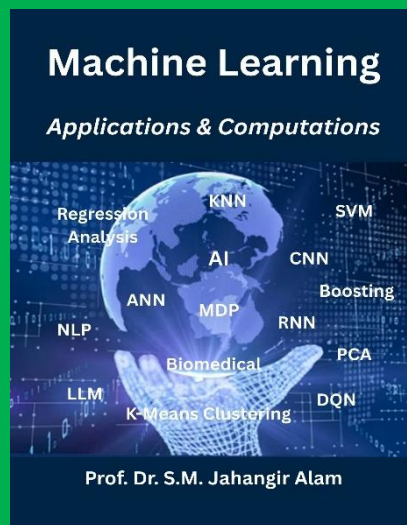
Final predicted class = 1

Summary:

- Bagging creates multiple bootstrap samples.
- Train base learners on each sample.
- Each learner predicts independently.
- Aggregate predictions (majority vote).
- Bagging reduces variance and improves accuracy.

Lecture 5.3

Ensemble Learning Learning



Instructor Name:
Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

K-fold Data Split K-Fold Cross Validation (Data Split)

- It is a model evaluation technique that helps estimate how well a model will generalize to unseen data.
- Instead of one fixed train-test split, we split the dataset into K equal parts ("folds") and train/test the model multiple times.

K-fold Data Split K-Fold Cross Validation (Data Split)

Steps of K-Fold:

1. Shuffle the dataset randomly.
2. Split into K equally (or nearly equally) sized folds.
3. For each fold i :
 - Use fold i as the test set.
 - Use the remaining $K - 1$ folds as the training set.
 - Train the model, compute the performance score.
4. Average the scores across all K folds $\rightarrow$ final model performance.

K-fold Data Split K-Fold Cross Validation (Data Split)

Advantages:

- Uses all data points for training and testing → less bias.
- More reliable estimate of model performance than single train/test split.

Disadvantages:

- Computationally expensive (model is trained K times).

Special Variants:

- Stratified K-Fold → preserves class proportions in each fold (used in classification).
- Leave-One-Out (LOO) → extreme case where $K = N$ (each fold has 1 sample).

K-fold Data Split K-Fold Cross Validation (Data Split)

Suppose we have **10 data points** with corresponding labels:
We want to do **5-Fold Cross-Validation** using **simple linear regression**.

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Step 1: Split the data into K=5 folds

Each fold will have 2 points:

- Fold 1: (1,2), (2,4)
- Fold 2: (3,6), (4,8)
- Fold 3: (5,10), (6,12)
- Fold 4: (7,14), (8,16)
- Fold 5: (9,18), (10,20)

K-fold Data Split K-Fold Cross Validation (Data Split)

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Step 2: Perform training/testing for each fold

Fold 1 (Test) → Folds 2-5 (Train)

- Training Data: (3,6),(4,8),(5,10),(6,12),(7,14),(8,16),(9,18),(10,20)
- Test Data: (1,2),(2,4)

Fit linear regression on training data: Simple pattern: $Y = 2X$

- Predictions on test data:

$$X = 1 \rightarrow \hat{Y} = 2 \times 1 = 2$$

$$X = 2 \rightarrow \hat{Y} = 2 \times 2 = 4$$

- **Error:** Both predictions match exactly → Error = 0.

K-fold Data Split K-Fold Cross Validation (Data Split)

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Fold 2 (Test) → Folds 1, 3-5 (Train)

Training Data: (1,2),(2,4),(5,10),(6,12),(7,14),(8,16),(9,18),(10,20)

Test Data: (3,6),(4,8)

Fit linear regression on training data: Simple pattern: $\hat{Y} = 2X$

- Predictions on test data:

$$X = 3 \rightarrow \hat{Y} = 6$$

$$X = 4 \rightarrow \hat{Y} = 8$$

- **Error:** Both predictions match exactly → Error = 0.

K-fold Data Split K-Fold Cross Validation (Data Split)

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Fold 3 (Test) → Folds 1, 2, 4, 5 (Train)

Training Data: (1,2),(2,4),(3,6),(4,8),(7,14),(8,16),(9,18),(10,20)

Test Data: (5,10),(6,12)

Fit linear regression on training data: Simple pattern: $\hat{Y} = 2X$

- Predictions on test data:

$$X = 5 \rightarrow \hat{Y} = 10$$

$$X = 6 \rightarrow \hat{Y} = 12$$

- **Error:** Both predictions match exactly → Error = 0.

K-fold Data Split K-Fold Cross Validation (Data Split)

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Fold 4 (Test) → Folds 1-3, 5 (Train)

Training Data: (1,2),(2,4),(3,6),(4,8),(5,10),(6,12),(9,18),(10,20)

Test Data: (7,14),(8,16)

Fit linear regression on training data: Simple pattern: $\hat{Y} = 2X$

• Predictions on test data:

$$X = 7 \rightarrow \hat{Y} = 14$$

$$X = 8 \rightarrow \hat{Y} = 16$$

Error: Both predictions match exactly → Error = 0.

K-fold Data Split K-Fold Cross Validation (Data Split)

X	1	2	3	4	5	6	7	8	9	10
Y	2	4	6	8	10	12	14	16	18	20

Fold 5 (Test) → Folds 1-4 (Train)

Training Data: (1,2),(2,4),(3,6),(4,8),(5,10),(6,12),(7,14),(9,18)

Test Data: (9,18),(10,20)

Fit linear regression on training data: Simple pattern: $\hat{Y} = 2X$

• Predictions on test data:

$$X = 9 \rightarrow \hat{Y} = 18$$

$$X = 10 \rightarrow \hat{Y} = 20$$

Error: Both predictions match exactly → Error = 0.

K-fold Data Split K-Fold Cross Validation (Data Split)

Step 3: Compute final metric

- Mean squared error (MSE) for each fold = 0
- Average MSE across all folds = 0

Result: Model perfectly predicts $Y = 2X$

Key Takeaways:

- Each data point is used exactly once as test data.
- Each fold trains on $K-1$ folds $\rightarrow$ reduces bias in performance estimation.
- Final performance metric: average over all folds.
- Works well even with small datasets.

Cross-validation to Test Accuracy

- When we train a machine learning model, we want to know:
How well will this model perform on unseen data?
- Instead of using just one train/test split (which might give a biased estimate), **Cross-Validation** provides a more reliable way to test accuracy.
- Cross-validation tests model accuracy by **repeatedly splitting data**, training, and testing, then averaging the results.

Cross-validation to Test Accuracy

Process:

1. **Split the dataset** into K folds (e.g., 5 or 10).
2. **Train on $K-1$ folds, test on the remaining fold.**
3. **Repeat for all folds** → each fold is tested once.
4. Compute **accuracy for each fold.**
5. **Average accuracies** → final accuracy estimate.

Why CV Accuracy is Better:

- Uses all data for training and testing.
- Reduces variance from lucky/unlucky splits.
- More trustworthy measure of generalization than a single train-test split.

Cross-validation to Test Accuracy

5.10##Perform cross-validation to test accuracy to test model accuracy using a small dataset and a simple classifier.

Goal: Estimate model accuracy using k-fold cross-validation.

Scenario: We have a dataset to predict if a person is Fit (1) or Unfit (0) based on their age.

ID	1	2	3	4	5
Age	25	45	30	50	35
Fit	1	0	1	0	1

Cross-validation to Test Accuracy

Step 1: Choose $k = 5$ (5-fold cross-validation):

We'll split the data into 5 folds. For each fold:

- Use 4 samples to train
- Test on the remaining 1
- Repeat 5 times (each sample gets to be the test once)

Step 2: Perform Each Fold (Using a Simple Rule:

If $\text{Age} \leq 40 \rightarrow \text{Predict Fit (1)}; \text{Else} \rightarrow \text{Unfit (0)}$)

Fold 1: Test = ID 1 (25, actual = 1)

Train = ID 2–5

Prediction: $25 \leq 40 \rightarrow \text{Predict 1} \rightarrow \text{Correct}$

ID	1	2	3	4	5
Age	25	45	30	50	35
Fit	1	0	1	0	1

Cross-validation to Test Accuracy

Fold 2: Test = ID 2 (45, actual = 0)

Train = IDs 1,3,4,5

Prediction: $45 > 40 \rightarrow$ Predict 0 $\rightarrow$ Correct

Fold 3: Test = ID 3 (30, actual = 1)

Train = ID 1,2,4,5

Prediction: $30 \leq 40 \rightarrow$ Predict 1 $\rightarrow$ Correct

Fold 4: Test = ID 4 (50, actual = 0)

Train = ID 1,2,3,5

Prediction: $50 > 40 \rightarrow$ Predict 0 $\rightarrow$ Correct

Fold 5: Test = ID 5 (35, actual = 1)

Train = ID 1–4

Prediction: $35 \leq 40 \rightarrow$ Predict 1 $\rightarrow$ Correct

ID	1	2	3	4	5
Age	25	45	30	50	35
Fit	1	0	1	0	1

Cross-validation to Test Accuracy

Step 2: Perform Each Fold (Using a Simple Rule:

If Age $\leq 40 \rightarrow$ Predict Fit (1); Else $\rightarrow$ Unfit (0))

Step 3: Calculate Accuracy

- Total predictions = 5
- Correct = 5
- Accuracy = $5/5 = 100\%$

ID	1	2	3	4	5
Age	25	45	30	50	35
Fit	1	0	1	0	1

Final Report: Using 5-fold cross-validation,

- Model predicted correctly in all 5 folds
- Estimated accuracy: 100%
- Rule used: If Age $\leq 40 \rightarrow$ Fit (1); Else $\rightarrow$ Unfit (0)

The model generalizes well on this tiny dataset.

Quantum Machine Learning (QML)

QML → Speed up ML training.

Quantum Machine Learning combines quantum computing with machine learning to create algorithms that can learn patterns in data, often faster or more efficiently than classical ML.

- Instead of classical bits (0/1) → QML uses qubits that can be in superposition (0 and 1 at the same time).
- Algorithms leverage entanglement and quantum parallelism to process data in higher dimensions.

Quantum Machine Learning uses qubits, superposition, entanglement, and quantum gates to speed up classification, clustering, regression, and deep learning.

It is still in early stages but has huge potential for future AI.

Quantum Machine Learning (QML)

- **Qubit (Quantum Bit)**

- Like a bit, but can be in **0, 1, or both at once**.

- Example: A qubit state:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, |\alpha|^2 + |\beta|^2 = 1$$

- **Superposition**

- A qubit can be in multiple states simultaneously.

- Example: Coin spinning in the air → both heads & tails until measured.

- **Entanglement**

- Two qubits can be **linked** so that measuring one instantly gives info about the other.

Quantum Machine Learning (QML)

- **Quantum Gates**
 - Operations that change qubit states (like logic gates in classical computing).
 - Examples: Hadamard (H), Pauli-X (like NOT), CNOT (entangles two qubits).
- **Measurement**
 - When measured, qubits collapse into classical 0 or 1, with certain probabilities.

Quantum Machine Learning (QML)

Example (Single Qubit)

- Start with qubit $|0\rangle$.
- Apply **Hadamard Gate (H)**:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

- Meaning:
 - 50% chance $\rightarrow 0$
 - 50% chance $\rightarrow 1$
- The qubit is now in **superposition**.

Quantum Machine Learning (QML)

Example (Entanglement)

- Start: $|00\rangle$ (two qubits both in 0).
- Apply Hadamard to first qubit:

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

- Apply CNOT $\rightarrow$ entangles them:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Now:

- If first qubit = 0 $\rightarrow$ second qubit is 0
- If first qubit = 1 $\rightarrow$ second qubit is 1

(This is a **Bell state**).

Quantum Machine Learning (QML)

Core Ideas of QML

- **Quantum Data Encoding**
 - Map classical data into quantum states.
 - Example: A data point (x_1, x_1) can be encoded as a qubit rotation state.
- **Quantum Operations (Quantum Gates)**
 - Like weights in neural networks, gates transform qubit states.
- **Measurement**
 - After computation, qubits collapse to classical outcomes (0/1).
 - Probabilities of outcomes are used for predictions.

Quantum Machine Learning (QML)

Mini Example (Quantum Classification)

Let's classify points using 1 qubit:

- Data point: $x = 0.6$
- Encode into qubit using a rotation gate:

$$|\psi\rangle = \cos\left(\frac{x}{2}\right) |0\rangle + \sin\left(\frac{x}{2}\right) |1\rangle$$

For $x = 0.6$:

$$|\psi\rangle = \cos(0.3) |0\rangle + \sin(0.3) |1\rangle = 0.955 |0\rangle + 0.295 |1\rangle$$

- Measurement:
 - Probability (0) $\approx 0.91 \rightarrow$ Class A
 - Probability (1) $\approx 0.09 \rightarrow$ Class B

Quantum circuit acts like a classifier.

Applications of QML

- **Healthcare:** Protein folding, drug discovery
- **Finance:** Risk modeling, fraud detection
- **Computer Vision:** Quantum CNNs for image recognition
- **NLP:** Quantum Transformers for text classification
- **Optimization:** Faster training for deep learning

Quantum Machine Learning (QML)

Mini Example (Quantum Classification)

Let's classify points using 1 qubit:

- Data point: $x = 0.6$
- Encode into qubit using a rotation gate:

$$|\psi\rangle = \cos\left(\frac{x}{2}\right) |0\rangle + \sin\left(\frac{x}{2}\right) |1\rangle$$

For $x = 0.6$:

$$|\psi\rangle = \cos(0.3) |0\rangle + \sin(0.3) |1\rangle = 0.955 |0\rangle + 0.295 |1\rangle$$

- Measurement:
 - Probability (0) $\approx 0.91 \rightarrow$ Class A
 - Probability (1) $\approx 0.09 \rightarrow$ Class B

Quantum circuit acts like a classifier.

Thank You